

AI Sales Training Platform

Technical Documentation

Backend API Reference & Developer Guide

Platform	AI Sales Training Platform
Framework	FastAPI (Python 3.9+)
AI Models	OpenAI GPT-4o-mini · Whisper · ElevenLabs
Database	MongoDB (Motor async)
Storage	AWS S3
Version	1.0.0
Status	Production Ready

© 2025 AI Sales Training Platform · Confidential

AI Sales Training Platform — Backend

A **FastAPI-based backend** for an AI-powered sales training platform. Salespeople practice live voice conversations with AI company representatives powered by **OpenAI GPT-4o-mini** and **ElevenLabs Voice AI**.

Table of Contents

- [Project Overview](#project-overview)
 - [Tech Stack](#tech-stack)
 - [Architecture](#architecture)
 - [Installation & Setup](#installation--setup)
 - [Environment Variables](#environment-variables)
 - [Running the Server](#running-the-server)
 - [API Reference](#api-reference)
 - [Health Check](#health-check)
 - [Salesperson APIs](#salesperson-apis)
 - [Company APIs](#company-apis)
 - [Meeting APIs](#meeting-apis)
 - [Conversation APIs](#conversation-apis)
 - [WebSocket — Live Conversation](#websocket--live-conversation)
 - [Admin APIs](#admin-apis)
 - [WebSocket Event Reference](#websocket-event-reference)
 - [Database Schema](#database-schema)
 - [Enums & Valid Values](#enums--valid-values)
 - [Flow — How It All Works](#flow--how-it-all-works)
 - [Audio Pipeline](#audio-pipeline)
 - [Error Handling](#error-handling)
-

Project Overview

This platform simulates real sales meetings. A salesperson speaks via microphone, the backend transcribes their speech (Whisper), generates an AI representative response (GPT-4o-mini), converts it to voice (ElevenLabs), and streams the audio back in real time over WebSocket.

Key Features

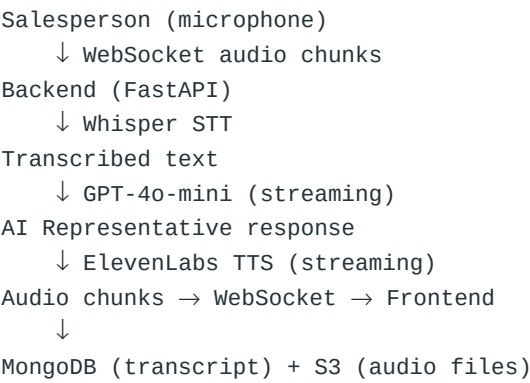
- Real-time voice conversation over WebSocket
- 1-on-1, 1-on-2, or 1-on-3 meeting modes
- Multiple AI representative personalities (angry, arrogant, soft, analytical, etc.)

- Auto-generated top 5 strategic questions per meeting
- Auto-scraped company data from website URL
- Full conversation transcript + audio saved to MongoDB + S3
- Per-session analytics (talk ratio, questions asked, engagement score)
- Sales methodology support (MEDDIC, BANT, Challenger, etc.)
- Admin system prompt override
- Salesperson AI insights dashboard

Tech Stack

Layer	Technology
Framework	FastAPI (Python 3.9+)
AI Responses	OpenAI GPT-4o-mini
Speech-to-Text	OpenAI Whisper
Text-to-Speech	ElevenLabs (streaming)
Database	MongoDB (Motor async)
File Storage	AWS S3
Web Scraping	BeautifulSoup4, httpx
Real-time	WebSocket (FastAPI native)

Architecture



Installation & Setup

Prerequisites

- Python 3.9+
- MongoDB (local or Atlas)
- AWS S3 bucket
- OpenAI API key
- ElevenLabs API key

Install

```
python -m venv venv
venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

Environment Variables

Create a `.env` file in the project root:

```
# OpenAI
OPENAI_API_KEY=sk-your-openai-key

# ElevenLabs
ELEVENLABS_API_KEY=your-elevenlabs-key

# AWS S3
AWS_ACCESS_KEY_ID=your-access-key
AWS_SECRET_ACCESS_KEY=your-secret-key
AWS_REGION=ap-south-1
S3_BUCKET_NAME=salesman-practice

# MongoDB
MONGODB_URL=mongodb://localhost:27017
MONGODB_DB_NAME=julienmts
```

Running the Server

```
uvicorn main:app --reload
```

- API base URL: <http://localhost:8000>
- Swagger docs: <http://localhost:8000/docs>
- Health check: <http://localhost:8000/health>

API Reference

All endpoints return this standard response shape:

```
{
  "success": true,
  "data": { ... },
  "message": "... "
}
```

Health Check

GET /

Returns API status.

GET /health

```
{ "status": "healthy", "database": "connected" }
```

Salesperson APIs

Base prefix: **/api/salesperson**

Router mounted at: **/salespersons**

Full path example: **POST /salespersons/api/salesperson/with-files**

POST /salespersons/api/salesperson/with-files

Create a salesperson profile with optional file uploads.

Content-Type: **multipart/form-data**

Field	Type	Required	Description
`product_name`	string	■	Name of the product being sold
`description`	string	■	Product description
`product_url`	string	■	Product website URL
`materials`	file[]	■	PDF, PPTX, DOC, JPG, PNG files

Response:

```
{
  "success": true,
  "data": {
    "salesperson_id": "abc123",
    "product_name": "CloudSync Pro",
    "materials_uploaded": 2
  }
}
```

```
}

```

GET /salespersons/api/salesperson/{salesperson_id}

Get salesperson profile by ID.

Response:

```
{
  "data": {
    "id": "abc123",
    "product_name": "CloudSync Pro",
    "product_url": "https://cloudsync.io",
    "description": "Enterprise cloud solution",
    "materials": [
      { "file_name": "deck.pptx", "file_url": "https://s3.../...", "file_type": "pptx" }
    ],
    "created_at": "2025-01-01T10:00:00Z"
  }
}
```

PUT /salespersons/api/salesperson/{salesperson_id}

Update salesperson profile. All fields optional.

Content-Type: `multipart/form-data`

Field	Type	Description
<code>`product_name`</code>	string	New product name
<code>`description`</code>	string	New description
<code>`product_url`</code>	string	New URL
<code>`materials`</code>	file[]	New files (appended to existing)

DELETE /salespersons/api/salesperson/{salesperson_id}

Delete salesperson profile.

GET /salespersons/api/salesperson/{salesperson_id}/ai-insights

Get AI-generated performance insights for a salesperson based on all completed meetings.

Response:

```
{
  "data": {
    "strength": "You ask good discovery questions.",
    "improvement": "Work on closing techniques.",
    "pattern": "You tend to talk too much in demos.",
  }
}
```

```
"dashboard_stats": {
  "total_meetings": 12,
  "people_met": 8,
  "success_rate": 66.7,
  "talk_to_listen_ratio": "45/55",
  "average_preparation_time": 72
},
"talk_ratio_by_type": {
  "Discovery": "42% / 58%",
  "Demo": "55% / 45%",
  "Closing": "N/A",
  "Follow-up": "N/A"
}
}
```

GET /salespersons/api/salesperson/ai-insights
Same as above but for the most recently updated salesperson (no ID needed).

GET /salespersons/api/salesperson/health
Health check for salesperson service.

Company APIs

Base prefix: **/api/company**
Router mounted at: **/companies**
Full path example: **POST /companies/api/company/create**

POST /companies/api/company/create
Create a company profile. Automatically scrapes the website for data.

Content-Type: **application/json**

```
{
  "company_url": "https://example.com",
  "salesperson_id": "abc123",
  "auto_fetch": true
}
```

Field	Type	Required	Description
`company_url`	string (URL)	■	Company website
`salesperson_id`	string	■	ID of the salesperson this company belongs to

Field	Type	Required	Description
`auto_fetch`	boolean	■	Default `true`. Scrapes company data automatically

Response:

```
{
  "data": {
    "company_id": "xyz789",
    "salesperson_id": "abc123",
    "company_data": {
      "industry": "Technology",
      "company_size": "200-500",
      "revenue": "$50M",
      "headquarters": "San Francisco",
      "wappalyzer_tech_stack": ["React", "AWS"],
      "latest_news": ["Company raises $10M Series A"]
    }
  }
}
```

GET /companies/api/company/{company_id}

Get company data by ID.

Response:

```
{
  "data": {
    "id": "xyz789",
    "salesperson_id": "abc123",
    "company_url": "https://example.com",
    "company_data": { ... },
    "created_at": "...",
    "last_updated": "..."
  }
}
```

POST /companies/api/company/{company_id}/representatives

Add one or more representatives to a company.

Content-Type: application/json

Body: Array of representative objects.

```
[
  {
    "name": "John Smith",
    "role": "CTO",
    "is_decision_maker": true,
    "linkedin_profile": "https://linkedin.com/in/johnsmith",
```

```
    "notes": "Very technical, asks hard questions",
    "voice_id": "elevenlabs_voice_id_here"
  }
]
```

Field	Type	Required	Description
`name`	string	■	Representative's full name
`role`	string	■	Job title (free text — CEO, CMO, CTO, VP Sales, etc.)
`is_decision_maker`	boolean	■	Default `false`
`linkedin_profile`	string (URL)	■	LinkedIn URL
`notes`	string	■	Behaviour notes for AI
`voice_id`	string	■	ElevenLabs voice ID

Response:

```
{
  "data": {
    "representative_ids": ["rep1", "rep2"]
  }
}
```

GET /companies/api/company/{company_id}/representatives

Get all representatives for a company.

Response:

```
{
  "data": {
    "representatives": [
      {
        "id": "rep1",
        "name": "John Smith",
        "role": "CTO",
        "is_decision_maker": true,
        "notes": "Very technical",
        "voice_id": "voice_abc"
      }
    ]
  }
}
```

PUT /companies/api/company/representatives/{rep_id}

Update a representative.

Content-Type: application/json

Same fields as create (single object, not array).

DELETE /companies/api/company/representatives/{rep_id}

Delete a representative.

GET /companies/api/company/{company_id}/account-details

Get full account details — company info, representatives, all meetings with analytics, and AI insights.

Response:

```
{
  "data": {
    "company_name": "Acme Corp",
    "company": {
      "id": "xyz789",
      "salesperson_id": "abc123",
      "company_url": "https://acme.com",
      "company_data": { ... }
    },
    "representatives": [
      { "id": "...", "name": "John", "role": "CTO", "is_decision_maker": true }
    ],
    "total_meetings": 3,
    "meetings": [
      {
        "meeting_id": "...",
        "salesperson_id": "abc123",
        "session_id": "...",
        "meeting_goal": "Close $50k deal",
        "status": "completed",
        "created_at": "...",
        "total_duration_seconds": 1800,
        "score": 78,
        "score_label": "Good",
        "analytics": {
          "total_turns": 24,
          "salesperson_talk_time": 450.0,
          "representatives_talk_time": 510.0,
          "total_duration": 960.0,
          "salesperson_talk_ratio": 46.88,
          "questions_asked": 7
        }
      }
    ],
    "ai_insights": {
      "average_engagement_score": 72,
      "engagement_label": "Good – Actively Engaged",
      "sentiment_trend": "improving",
      "sentiment_trend_label": "Getting better over time",
      "risk_alerts": [
        { "type": "warning", "message": "Salesperson talks too much in demos" }
      ],
    },
  },
}
```

```
    "upsell_opportunities": [
      { "title": "Premium Plan", "reason": "Company is growing fast" }
    ]
  }
}
```

Meeting APIs

Base prefix: `/api/meeting`

Router mounted at: `/meetings`

Full path example: `POST /meetings/api/meeting/create`

POST `/meetings/api/meeting/create`

Create a new meeting. Validates all IDs and auto-generates top 5 strategic questions.

Content-Type: `application/json`

```
{
  "salesperson_id": "abc123",
  "company_id": "xyz789",
  "meeting_mode": "1-on-1",
  "representatives": ["rep1"],
  "meeting_goal": "Close a $50k deal for CloudSync Pro",
  "personality": "nice",
  "duration_minutes": 30,
  "difficulty": "intermediate",
  "sales_methodology": "MEDDIC",
  "custom_sales_methodology": null,
  "methodology_description": "Focus on economic buyer identification"
}
```

Field	Type	Required	Description
<code>`salesperson_id`</code>	string	■	Must exist in DB
<code>`company_id`</code>	string	■	Must exist in DB
<code>`meeting_mode`</code>	string	■	<code>`1-on-1`</code> / <code>`1-on-2`</code> / <code>`1-on-3`</code>
<code>`representatives`</code>	string[]	■	Must match meeting_mode count
<code>`meeting_goal`</code>	string	■	Describe what the salesperson wants to achieve
<code>`personality`</code>	string	■	Meeting-level personality. Default: <code>`nice`</code>

Field	Type	Required	Description
`duration_minutes`	integer	■	Default: `30`
`difficulty`	string	■	`"beginner"` / `"intermediate"` / `"advanced"` / `"expert"`
`sales_methodology`	string	■	See [Enums](#enums--valid-values). Default: <code>"MEDDIC"</code>
`custom_sales_methodology`	string	■	Custom name when <code>sales_methodology = "Other"</code>
`methodology_description`	string	■	Extra context for AI behavior

Response:

```
{
  "data": {
    "meeting_id": "meet123",
    "meeting_mode": "1-on-1",
    "meeting_personality": "nice",
    "duration_minutes": 30,
    "difficulty": "intermediate",
    "top_5_questions": [
      "What are your current challenges in this area?",
      "How are you handling this today?",
      "What would success look like for you?",
      "What is your timeline for a decision?",
      "Who else is involved in this decision?"
    ],
    "representatives": [
      { "id": "rep1", "name": "John Smith", "role": "CTO", "is_decision_maker": true }
    ],
    "status": "pending"
  }
}
```

GET /meetings/api/meeting/{meeting_id}

Get meeting details including latest session ID.

Response:

```
{
  "data": {
    "id": "meet123",
    "salesperson_id": "abc123",
    "company_id": "xyz789",
    "meeting_mode": "1-on-1",
    "representative_ids": ["rep1"],
    "meeting_goal": "...",
  }
}
```

```
"top_5_questions": [...],
"personality": "nice",
"duration_minutes": 30,
"difficulty": "intermediate",
"sales_methodology": "MEDDIC",
"status": "pending",
"session_id": "sess_latest_or_null",
"started_at": null,
"ended_at": null,
"total_duration_seconds": 0,
"representatives": [ { "id": "...", "name": "...", "role": "..." } ]
}
}
```

POST /meetings/api/meeting/{meeting_id}/start

Start a meeting. Changes status from **pending** (or **completed**) → **active**.

Response:

```
{ "success": true, "message": "Meeting started successfully" }
```

POST /meetings/api/meeting/{meeting_id}/end

End an active meeting. Changes status → **completed**. Calculates total duration.

Response:

```
{
  "success": true,
  "data": { "duration_seconds": 1842.5 },
  "message": "Meeting ended successfully"
}
```

DELETE /meetings/api/meeting/{meeting_id}

Delete a meeting.

GET /meetings/api/meeting/salesperson/{salesperson_id}/meetings

Get all meetings for a salesperson.

Response:

```
{
  "data": {
    "meetings": [ { ... }, { ... } ],
    "total": 5
  }
}
```

Conversation APIs

Base prefix: `/api/conversation`

Router mounted at: `/conversations`

Full path example: `POST /conversations/api/conversation/send-message`

`POST /conversations/api/conversation/send-message`

Send a text message and get AI response (non-WebSocket, HTTP).

Query Parameters:

Param	Type	Required	Description
<code>`meeting_id`</code>	string	■	Active meeting ID
<code>`speaker`</code>	string	■	Default <code>`salesperson`</code>
<code>`message`</code>	string	■	Message text

Optional: `audio_data` file upload.

Response:

```
{
  "data": {
    "primary_response": {
      "speaker_id": "rep1",
      "speaker_name": "John Smith",
      "speaker_role": "CTO",
      "response_text": "That's interesting. What's your pricing?",
      "audio_url": "https://s3.../audio.mp3",
      "audio_base64": "base64encodedaudio...",
      "audio_mime_type": "audio/mpeg",
      "turn_number": 2
    },
    "secondary_response": null,
    "salesperson_turn": 1,
    "reasoning": "CTO responds to technical question"
  }
}
```

`GET /conversations/api/conversation/{meeting_id}/sessions`

Get all practice sessions for a meeting, newest first.

Response:

```
{
  "data": {
    "sessions": [
      {
```

```
    "session_id": "sess1",
    "attempt_number": 2,
    "total_turns": 18,
    "created_at": "...",
    "recording_url": "https://s3.../presigned-url"
  }
]
}
```

GET /conversations/api/conversation/{meeting_id}/sessions/{session_id}

Get full transcript for a specific session.

Response:

```
{
  "data": {
    "session_id": "sess1",
    "attempt_number": 1,
    "turns": [
      {
        "turn_number": 1,
        "speaker": "salesperson",
        "speaker_name": "Salesperson",
        "text": "Hi, I wanted to discuss...",
        "audio_url": null,
        "duration_seconds": 5.0
      },
      {
        "turn_number": 2,
        "speaker": "rep1",
        "speaker_name": "John Smith",
        "text": "Go ahead, what is it about?",
        "audio_url": "https://s3.../...",
        "duration_seconds": 6.0
      }
    ],
    "total_turns": 2,
    "salesperson_talk_time": 5.0,
    "representatives_talk_time": 6.0,
    "analytics": { ... }
  }
}
```

GET /conversations/api/conversation/{meeting_id}/sessions/{session_id}/stream-audio

Stream the full session recording as audio.

Returns a streaming audio response (MP3).

GET /conversations/api/conversation/{meeting_id}/analytics

Get analytics for the most recent session of a meeting.

Response:

```
{
  "data": {
    "total_turns": 24,
    "salesperson_turns": 12,
    "ai_turns": 12,
    "salesperson_talk_time": 450.0,
    "representatives_talk_time": 510.0,
    "total_duration": 960.0,
    "salesperson_talk_ratio": 46.88,
    "representatives_talk_ratio": 53.13,
    "questions_asked": 7,
    "overall_score": 74,
    "engagement_score": 68
  }
}
```

WebSocket — Live Conversation

WS ws://localhost:8000/conversations/api/conversation/ws/live-conversation/{meeting_id}

The meeting must be **active** before connecting.

See [WebSocket Event Reference](#websocket-event-reference) for all message types.

Admin APIs

Base prefix: /api/admin

Router mounted at: /admin

GET /admin/api/admin/system-prompt

Get the current global admin system prompt.

PUT /admin/api/admin/system-prompt

Set a global system prompt that overrides all AI behaviour.

```
{ "prompt": "You are a hostile CFO. Be very resistant to any pitch." }
```

Set to empty string "" to disable and revert to default behaviour.

GET /admin/api/admin/methodology-prompts

Get all sales methodology prompts.

PUT /admin/api/admin/methodology-prompts/{methodology_id}

Update a methodology prompt (e.g. **MEDDIC**, **BANT**, etc.).

```
{ "prompt": "The rep is using MEDDIC. Make them identify the economic buyer." }
```

WebSocket Event Reference

Client → Server (messages you send)

audio_chunk — Send audio while speaking

```
{
  "type": "audio_chunk",
  "data": "base64encodedaudiochunk",
  "is_speaking": true
}
```

Send **is_speaking: false** on the last chunk to signal end of speech.

ping — Keepalive

```
{ "type": "ping" }
```

disconnect — Close session

```
{ "type": "disconnect" }
```

Server → Client (messages you receive)

connected — Connection established

```
{
  "type": "connected",
  "meeting_id": "meet123",
  "session_id": "sess456",
  "attempt_number": 1,
  "meeting_mode": "1-on-1",
  "duration_minutes": 30,
  "difficulty": "intermediate",
  "meeting_personality": "nice",
  "representatives": [
    {
      "id": "rep1",
```

```
    "name": "John Smith",
    "role": "CTO",
    "personality": ["nice"],
    "is_decision_maker": true
  }
]
```

transcription — Your speech was transcribed

```
{
  "type": "transcription",
  "text": "What is your current tech stack?",
  "speaker": "salesperson"
}
```

transcription_empty — Audio too short / silent

```
{
  "type": "transcription_empty",
  "message": "Audio was too short or silent. Please speak clearly and try again."
}
```

ai_thinking — AI is generating response

```
{
  "type": "ai_thinking",
  "message": "John Smith is preparing to speak..."
}
```

ai_text_token — Streaming text token (real-time)

```
{
  "type": "ai_text_token",
  "token": "That",
  "speaker_name": "John Smith",
  "speaker_role": "CTO"
}
```

ai_text_done — Full AI text ready

```
{
  "type": "ai_text_done",
  "full_text": "That's a good question. Let me think about it.",
  "speaker_name": "John Smith",
  "speaker_role": "CTO",
  "speaker_id": "rep1"
}
```

audio_chunk — Streaming audio from AI

```
{
  "type": "audio_chunk",
  "audio": "base64encodedaudiochunk",
  "speaker_name": "John Smith"
}
```

```
}
```

audio_done — AI audio stream complete

```
{
  "type": "audio_done",
  "speaker_name": "John Smith"
}
```

conversation_saved — Turn saved to DB

```
{
  "type": "conversation_saved",
  "turn_number": 4
}
```

error — Something went wrong

```
{
  "type": "error",
  "message": "Speech recognition failed: timeout"
}
```

pong — Response to ping

```
{ "type": "pong" }
```

Database Schema

Collection: salespeople

```
{
  "_id": "uuid",
  "product_name": "CloudSync Pro",
  "product_url": "https://cloudsync.io",
  "description": "Enterprise cloud storage",
  "materials": [
    { "file_name": "deck.pptx", "file_url": "https://s3/...", "file_type": "pptx" }
  ],
  "created_at": "datetime",
  "updated_at": "datetime"
}
```

Collection: companies

```
{
  "_id": "uuid",
  "salesperson_id": "uuid",
  "company_url": "https://acme.com",
  "company_data": {
    "company_size": "200-500",

```

```
"headquarters": "San Francisco",
"revenue": "$50M",
"industry": "Technology",
"wappalyzer_tech_stack": ["React", "AWS", "MongoDB"],
"hiring_data": { "open_positions": 12, "hiring_summary": "..."},
"customer_reviews": { "rating": 4.2, "total_reviews": 320, "summary": "..."},
"latest_news": ["Company raises $10M"],
"financial_statements": { "yoy_growth": "35%", "arr": "$8M" },
"product_documentation": { "api_docs_available": true }
},
"created_at": "datetime",
"last_updated": "datetime"
}
```

Collection: representatives

```
{
  "_id": "uuid",
  "company_id": "uuid",
  "name": "John Smith",
  "role": "CTO",
  "is_decision_maker": true,
  "linkedin_profile": "https://linkedin.com/in/...",
  "notes": "Very technical, asks hard questions",
  "voice_id": "elevenlabs_voice_id",
  "created_at": "datetime"
}
```

Collection: meetings

```
{
  "_id": "uuid",
  "salesperson_id": "uuid",
  "company_id": "uuid",
  "meeting_mode": "1-on-1",
  "representative_ids": ["rep1"],
  "meeting_goal": "Close a $50k deal",
  "top_5_questions": ["Q1", "Q2", "Q3", "Q4", "Q5"],
  "personality": "nice",
  "duration_minutes": 30,
  "difficulty": "intermediate",
  "sales_methodology": "MEDDIC",
  "methodology_description": "Focus on economic buyer",
  "status": "pending",
  "created_at": "datetime",
  "started_at": null,
  "ended_at": null,
  "total_duration_seconds": 0,
  "expected_end_time": null
}
```

Collection: conversations

```
{
  "_id": "uuid",
```

```
"session_id": "unique_per_session",
"meeting_id": "uuid",
"attempt_number": 1,
"turns": [
  {
    "turn_number": 1,
    "speaker": "salesperson",
    "speaker_name": "Salesperson",
    "text": "Hi, I wanted to discuss...",
    "audio_url": null,
    "timestamp": "00:00:10",
    "duration_seconds": 5.0,
    "created_at": "datetime"
  },
  {
    "turn_number": 2,
    "speaker": "rep1",
    "speaker_name": "John Smith",
    "text": "Go ahead.",
    "audio_url": "https://s3.../turn_002.mp3",
    "duration_seconds": 3.0,
    "created_at": "datetime"
  }
],
"total_turns": 2,
"salesperson_talk_time": 5.0,
"representatives_talk_time": 3.0,
"recording_s3_url": "https://s3.../full_recording.mp3",
"analytics": { ... },
"created_at": "datetime"
}
```

Enums & Valid Values

meeting_mode

Value	Description
`"1-on-1"`	1 representative
`"1-on-2"`	2 representatives
`"1-on-3"`	3 representatives

personality

Value
`"angry"`
`"arrogant"`
`"soft"`

Value
`"cold_hearted"`
`"nice"`
`"cool"`
`"not_well"`
`"analytical"`
`"professional"`
`"casual"`
`"direct"`

difficulty

Value
`"beginner"`
`"intermediate"`
`"advanced"`
`"expert"`

sales_methodology

Value
`"MEDDIC"`
`"Challenger Sales"`
`"BANT"`
`"SPIN Selling"`
`"MEDDPICC"`
`"Value Selling"`
`"Other"`

When "Other" is selected, provide custom_sales_methodology string.

Flow — How It All Works

Step 1 — Create Salesperson

POST /salespersons/api/salesperson/with-files

- Upload product materials to S3
- Store profile in MongoDB
- Returns salesperson_id

Step 2 — Create Company

- POST /companies/api/company/create
- Scrape website for company data
 - Store in MongoDB with salesperson_id
 - Returns company_id

Step 3 — Add Representatives

- POST /companies/api/company/{company_id}/representatives
- Store rep profiles in MongoDB
 - Returns representative_ids

Step 4 — Create Meeting

- POST /meetings/api/meeting/create
- Validates salesperson, company, reps
 - Validates meeting_mode vs rep count
 - Generates top 5 questions via GPT
 - Returns meeting_id + questions

Step 5 — Start Meeting

- POST /meetings/api/meeting/{meeting_id}/start
- Changes status: pending → active

Step 6 — Live Conversation (WebSocket)

WS /conversations/api/conversation/ws/live-conversation/{meeting_id}

1. Connect → receive "connected" event with rep details
2. Speak → send audio_chunk events (is_speaking: true)
3. Stop speaking → send audio_chunk (is_speaking: false)
4. Backend transcribes with Whisper
5. Backend streams GPT response token by token
6. ElevenLabs converts text → audio stream
7. Audio chunks sent back via "audio_chunk" events
8. Turn saved to MongoDB

Step 7 — End Meeting

- POST /meetings/api/meeting/{meeting_id}/end
- Changes status: active → completed
 - Calculates total duration

Step 8 — Review Results

GET /companies/api/company/{company_id}/account-details

→ Full meeting history + analytics + AI insights

GET /salespersons/api/salesperson/{salesperson_id}/ai-insights

→ Cross-meeting performance analysis

Audio Pipeline

Microphone → Browser MediaRecorder (WebM/Opus)

↓ base64 chunks over WebSocket

Backend audio_stream_service (buffers chunks)

↓ joined bytes

Whisper STT

↓ hallucination filter (MIN_AUDIO_BYTES=4000, known filler words)

Transcribed text

↓

GPT-4o-mini streaming (presence_penalty=0.8, frequency_penalty=0.8)

↓ token by token

ElevenLabs TTS streaming

↓ audio chunks

WebSocket → Frontend audio player

↓

S3 upload (full recording stitched together)

Hallucination Protection

Whisper sometimes returns false positives on short/silent audio. The backend discards:

- Audio below 4,000 bytes (~0.25 seconds)
- Results matching known filler patterns: "thank you", "bye", "okay", "uh", etc.

Error Handling

All endpoints return HTTP 4xx/5xx with:

```
{ "detail": "Error description here" }
```

Common codes:

Code	Meaning
`400`	Bad request (wrong meeting mode count, meeting not active, etc.)
`404`	Resource not found
`500`	Internal server error

WebSocket errors are sent as { "type": "error", "message": "..." } and the connection stays open.

Security Notes

- Never commit `.env` to version control
- Use IAM roles for AWS in production
- Add JWT authentication before going to production
- CORS is currently set to `allow_origins=["*"]` — restrict in production
- S3 audio files use presigned URLs for time-limited access